Project Name: Skill-It (Group 4)
Project Members: Braeden, Jeremy, Daniel, Madhav, Ryan

1. Gig / Jobs Object — Braeden

Internal Logical Files (ILFs) are database tables owned and maintained by our system.

Item	Complexity	FP
Job table	Average	10
Category table	Simple	7
Skill table	Simple	7
User Profile table	Average	10
Review table	Average	10

The Job table is average because it has 10+ fields and relationships to four other tables. Category and Skill are simple because they each have only 2–3 fields with no complex relationships. User Profile is average due to its 12+ fields and references to Skills and MessageThreads. Review is average because it references two UserProfile objects (From and To) plus an associated Job.

External Interface Files (EIFs) are data sources our system reads but does not own.

Item	Complexity	FP
Supabase Auth User	Simple	5

The Supabase Auth User table is owned and managed entirely by Supabase's authentication infrastructure. We read it only to verify user identity and never write to it directly, making it an EIF rather than an ILF.

External Inputs (EIs) are operations that enter the system and modify data.

Item	Complexity	FP
Create job posting (POST /api/jobs/)	Average	4
Edit job (PUT /api/jobs/{jobId})	Simple	3
Delete job (DELETE /api/jobs/{jobId})	Simple	3
Mark job complete (PATCH /api/jobs/{jobId}/complete)	Simple	3

Apply to job (POST /api/jobs/{jobId}/request)	Simple	3
Withdraw application (DELETE /api/jobs/{jobId}/request)	Simple	3
Accept a worker (POST /api/jobs/{jobId}/accept/{userId})	Average	4
Remove accepted worker (DELETE /api/jobs/{jobId}/accept/{userId})	Simple	3

Creating a job is average because it writes to the Job table while also referencing Category and Skill. Accepting a worker is average because it updates AcceptedWorkers and must validate the user against UserProfile. All other operations are simple because they each touch only one table with a single, straightforward update or delete.

External Outputs (EOs) are operations that produce computed or derived results. This feature has none — all data returned by the API is retrieved directly from the database with no transformation or calculation applied.

External Inquiries (EQs) are read-only operations that retrieve data without modifying anything.

Item	Complexity	FP
Filter jobs by category	Simple	3
Filter jobs by skills	Simple	3
Filter open/closed jobs	Simple	3
Jobs by user (GET /api/jobs/user/{userId})	Simple	3
Pending requests for a job (GET /api/jobs/{jobId}/requests)	Average	4
Reviews for a job (GET /api/jobs/{jobId}/reviews)	Average	4
Threads for a job (GET /api/jobs/{jobId}/threads)	Simple	3

The category, skills, open/closed, and user filters are all simple because they apply a single filter condition to one or two tables. Pending requests is average because it includes a poster-only authorization check and joins Job with UserProfile. Reviews is average because it joins Review with two separate UserProfile references (the reviewer and the reviewed). Threads is simple because it is a straightforward filter on the MessageThread table by Job foreign key.

Total: 98 Function Points

---

2. Message Threads / Message Text — Braeden

Internal Logical Files (ILFs) are database tables owned and maintained by our system.

Item	Complexity	FP
Message Thread table	Average	10
Message table	Simple	7
User Profile table	Average	10

MessageThread is average because it has 7 fields and references Users, Job, and Messages while also tracking archived and name state. Message is simple because it has only 4 fields. User Profile is average for the same reasons as in the Jobs feature — 12+ fields with multiple relationships.

External Interface Files (EIFs) are data sources our system reads but does not own.

Item	Complexity	FP
Supabase Auth User	Simple	5

Same reasoning as the Jobs feature — we read this only to verify who the logged-in user is, and Supabase owns and manages it entirely.

External Inputs (EIs) are operations that enter the system and modify data.

Item	Complexity	FP
Create thread (POST /api/threads/)	Average	4
Archive thread (PATCH /api/threads/{threadId}/archive)	Simple	3
Rename thread (PUT /api/threads/{threadId}/name)	Simple	3
Send message (POST /api/threads/{threadId}/messages)	Simple	3
Delete message (DELETE /api/threads/{threadId}/messages/{messageId})	Simple	3
Add participant (POST /api/threads/{threadId}/users/{userId})	Simple	3

Remove participant (DELETE /api/threads/{threadId}/users/{userId})	Simple	3
Mark messages as read (triggered on thread open)	Simple	3

Creating a thread is average because it creates a MessageThread record while simultaneously linking a Job and setting initial participants from UserProfile. All other operations are simple because they each perform a single, targeted update or delete on one table.

External Outputs (EOs) are operations that produce computed or derived results. This feature has none — all data returned is retrieved directly from the database with no transformation or calculation applied.

External Inquiries (EQs) are read-only operations that retrieve data without modifying anything.

Item	Complexity	FP
All threads for logged-in user (GET /api/threads/)	Average	4
Specific thread + messages (GET /api/threads/{threadId})	Average	4
Messages in thread (GET /api/threads/{threadId}/messages)	Simple	3
Participants in thread (GET /api/threads/{threadId}/users)	Simple	3
Threads for a job (GET /api/threads/job/{jobId})	Simple	3
Unread threads (GET /api/threads/unread)	Average	4

Retrieving all threads for a user is average because it joins MessageThread and UserProfile filtered by the current session. Retrieving a specific thread with its messages is average because it joins across three tables: MessageThread, Message, and UserProfile. Messages and participants are simple because each joins only two tables with a straightforward filter. Threads by job is simple — a single foreign key filter. Unread threads is average because it joins MessageThread, Message, and UserProfile while filtering by unread status across all three.

Total: 78 Function Points

Project Name: Skill-It (Group 4)
Project Members: Braeden, Jeremy, Daniel, Madhav, Ryan

3. Skills – Daniel

Internal Logical Files (ILFs) are database tables owned and maintained by our system.

Item	Complexity	FP
Skills table	Simple	7
Profile-Skill-Holder table	Average	10
Job-Skill-Holder table	Average	10

Skills are simple, just a primary key int, name, and explicit tag. The skill-holder tables are average complexity since they both have a primary key composed from two foreign keys.

External Inputs (EIs) are operations that enter the system and modify data.

Item	Complexity	FP
Create new skill (POST /api/skill/{skillName})	Simple	3
Make a skill explicit (PATCH /api/skill/{skillId}/explicit)	Simple	3
Add skill to a profile (POST /api/pskillHolder/{userId}/{skillId})	Average	4
Delete skill from a profile (DELETE /api/pskillHolder/{userId}/{skillId})	Simple	3
Add skill to a job (POST /api/jskillHolder/{jobId}/{skillId})	Average	4
Delete skill from a job (DELETE /api/jskillHolder/{jobId}/{skillId})	Simple	3

Creating a skill is simple because it only refers to the Skill table. Same goes for making a skill explicit. Adding a skill to a profile is more complex because it requires checking that the skill is not explicit first. Deleting a skill from a profile is straightforward. Adding and deleting skills from jobs has the same complexities as doing so for profiles.

External Outputs (EOs) are operations that produce computed or derived results. This feature has none — all data returned by the API is retrieved directly from the database with no transformation or calculation applied.

Project Name: Skill-It (Group 4)
Project Members: Braeden, Jeremy, Daniel, Madhav, Ryan

External Inquiries (EQs) are read-only operations that retrieve data without modifying anything.

Item	Complexity	FP
Get a skill by id (GET /api/skill/{skillId})	Simple	3
Get all skills from a profile (GET /api/pskillHolder/{userId})	Average	4
Get all skills from a job (GET /api/jskillHolder/{jobId})	Average	4
Get all profiles matching certain skills (GET /api/pskillHolder/{skillId}/profiles)	Complex	6
Get all jobs matching certain skills (GET /api/jskillHolder/{skillId}/jobs)	Complex	6

Getting a skill is super simple, just using the primary key to return the row. Getting all skills from a profile/job requires returning all rows matching a value in one column, so average. Getting all profiles or jobs matching multiple skills is complex, requiring multiple calls to the database, one for each skill. The function would then group by which jobs or profiles match the most skills from the input.

27 (ILF) + 20 (EI) + 0 (EO) + 23 (EQ) = 70

Total: 70 Function Points

Project Name: Skill-It (Group 4)
Project Members: Braeden, Jeremy, Daniel, Madhav, Ryan

4. Task Related to Reviews

Creating and Posting Reviews

Task breakdown: form renders on webpage → User fills out form and clicks submit → calls method reviews.createReviews → updates table

Internal Logical Files (ILFs) are database tables owned and maintained by our system.

Item	Complexity	FP
Review table	Simple	7
Profile-Review-Holder table	Average	10
Job-Review-Holder table	Average	10

To create and post reviews, we need to update the review table alongside the associated relational tables.

External Inputs (EIs) are operations that enter the system and modify data.

Item	Complexity	FP
Create a review (POST /api/reviews/)	Simple	3

This method would simply update the corresponding table with the

Total: 27 (ILFs)+3 (EIs)=30 Points

Retrieving and Viewing Reviews Based on Profile

Task breakdown: render webpage of profile → retrieves profile → retrieves review based on profile id → lists the profile and the review

Internal Logical Files (ILFs) are database tables owned and maintained by our system.

Item	Complexity	FP
Review table	Simple	7
Profile-Review-Holder table	Average	10
Profile Table	Average	10

These are corresponding tables needed to look up a profile, then look up corresponding reviews on that profile.

External Outputs (EOs) are operations that produce computed or derived results.

Item	Complexity	FP
Get a Review for user (GET /api/profile/{uid}/reviews/received)	Simple	3
Get a Profile (GET /api/profile)	Simple	3

The second method would retrieve the profile from the profile table, and then use the relation table to retrieve the review of the review table.

Total: 27(ILFS) + 6(EOs) = 33 Points

5. Categories - Madhav

Creating and Adding Categories

Project Name: Skill-It (Group 4)
Project Members: Braeden, Jeremy, Daniel, Madhav, Ryan

6. Account Creation & Profile - Ryan

Internal Logical Files (ILFs)

Item	Complexity	FP
Profile Table	Average	10
Skill Table	Simple	7

The profile table is classified as average because there are 12+ fields and many references to other tables. It is intended to be the core entity of the application. It is directly related to the Supabase Authentication Architecture.

External Interface Files (EIFs)

Item	Complexity	FP
Supabase Auth User	Simple	5

All user verification and authentication will be done using Supabase's built-in authentication system. We will not need to keep track of passwords or login credentials. Each user in this system will have a direct relationship to a profile in the application.

External Outputs (EOs) - This functionality does not require any external outputs. All accesses will be directly output from the system and require no calculations.

External Inputs (EIs)

Item	Complexity	FP
Account Creation	Simple	3
Login	Complex	6
Logout	Simple	3

Update Password	Complex	6
Create Profile	Simple	3
Edit Profile	Simple	3
Verify/Ban Profile	Simple	3
Add Skill	Average	4
Remove Skill	Simple	4

Most EIs in this module are simple. EIs classified as simple only have effect in the Profile table and perform simple CRUD operations. Adding a skill is listed as average as it may require updates in both the Profile and Skills tables. In the case of Login and Update Password, these functionalities directly use the Supabase Authentication Architecture and are classified as complex for this reason as well as inferring the importance of security for these functionalities.

External Inquiries (EQs)

Item	**Complexity**	**FP**
Callback	Complex	6
Get Profile by User	Simple	3
Get Profile by Skill	Simple	3

Getting a user profile by skill and userId are simple as they are only database accesses requiring no calculations. Callback is classified as complex because it requires a long chain of events ultimately handled by Supabase's authentication system. Callback should require little time implementing on our end however.

Total Function Points - 17 (ILFs) + 5 (EIF) + 35 (EIs) + 12 (EQs) = **69**